

Entrega Final — Ecosistema de Automatización con IA

Caso elegido: un sistema que recibe las consultas y reclamos de clientes por Gmail, les da una primera respuesta con IA, y antes de mandarla espera que yo (o alguien del equipo) la apruebe.

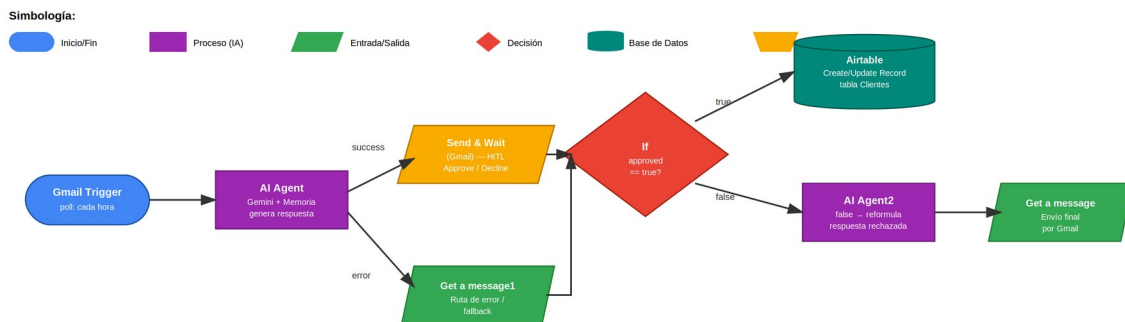
Alfonsina Coria Giardella

1. Por qué elegí este caso

Elegí automatizar la atención de consultas por mail porque es algo que me pasa todos los días en mi trabajo: entran correos de clientes con reclamos o preguntas y siempre hay que pararse a leer, redactar una respuesta y recién ahí mandarla. La idea del flujo es que esa primera lectura y el primer borrador de respuesta los haga la IA, pero que la decisión de mandarlo o no la siga tomando una persona. Por eso el punto de aprobación humana no es un detalle más, es el centro del diseño.

2. Cómo armé la arquitectura

El orquestador es n8n. Uso el trigger de Gmail revisando la casilla cada hora, y de ahí en adelante todo el flujo es automático hasta llegar al punto donde necesita mi aprobación. Para dibujar el diagrama usé la simbología típica de diagramas de flujo (óvalo para inicio, rectángulo para proceso, paralelogramo para entrada/salida, rombo para decisión y cilindro para base de datos) para que se entienda igual que un diagrama de programación normal, no como un dibujo genérico.



Memoria del sistema:

Simple Memory (buffer window, sessionKey = threadId de Gmail) inyectada en AI Agent → mantiene contexto por hilo de conversación.
Modelo IA: Google Gemini (mChatGoogleGemini) conectado a AI Agent y AI Agent2 via ai_languageModel.

Punto Human-in-the-loop:

"Send message and wait for response" pausa el flujo y envía botones Approve/Decline al operador (alfrodeyha@gmail.com) antes de tocar al cliente final. El nodo "If" evalúa \$json.data.approved para decidir la ruta.

Manejo de errores:

AI Agent: retryOnFail=true + onError="continueErrorOutput" → si falla la API de IA, no rompe el flujo; sale por el output de error hacia "Get a message1" en vez de detener la ejecución completa.

Así se recorre el flujo

- Llega un mail nuevo → el Gmail Trigger lo detecta (revisa cada hora).

- El AI Agent (con Gemini de motor y una memoria por hilo de conversación) lee el mail y escribe una respuesta corta, de máximo 300 caracteres.
- Si el AI Agent pudo generar la respuesta sin problemas, el flujo sigue por 'Send message and wait for response': le mando ese borrador a mi correo con dos botones, Approve o Decline, y ahí el flujo se queda esperando. No se manda nada al cliente todavía.
- Si el AI Agent falla (por ejemplo se cae la API de Gemini), en vez de romper todo el flujo, cae en una rama de error que va a 'Get a message1' — es la ruta de contingencia.
- Cuando yo respondo el mail de aprobación, el nodo If revisa si aprobé (true) o rechacé (false).
- Si aprobé: se guarda el registro en Airtable, con la consulta original y la respuesta que quedó aprobada.
- Si rechacé: pasa por un segundo agente (AI Agent2) que reformula la respuesta usando el mismo modelo, y ese nuevo intento vuelve a pasar por el circuito.
- Recién en 'Get a message' se manda finalmente el correo al cliente.

3. Cómo organicé los datos

Para que el sistema tenga memoria uso Airtable como base, con una tabla llamada 'Clientes' dentro de la base 'Gestión de Clientes FG'. La pensé simple, con lo mínimo necesario para poder seguir el estado de cada consulta:

Campo	Tipo	Para qué lo uso
Nombre	Texto único	Identificar de quién es la consulta
Estado de Consulta	Selección única: Pendiente / En Proceso / Respondido	Es el campo que me deja saber en qué punto está cada caso sin tener que leer todo de nuevo
Historial de Cambios	Texto largo	Ahí queda la respuesta que generó la IA y que yo aprobé
Validación de Mail (AI)	Selección única: Válido / Inválido	Un chequeo automático para detectar correos con formato raro
Consulta	Texto largo	El fragmento original del mail que llegó (lo que n8n llama 'snippet')

Lo importante de esta tabla es que no queda aislada: cada fila se relaciona con un hilo de Gmail (el threadId), que es el mismo identificador que uso como memoria en el AI Agent. Así, si el mismo cliente escribe otra vez, el sistema no arranca de cero.

Qué información viaja entre los nodos

Esto es básicamente lo que un nodo le pasa al siguiente, en formato JSON:

Del Gmail Trigger al AI Agent:

```
{ "id": "...", "threadId": "...", "snippet": "texto del mail", "Subject": "asunto" }
```

Lo que devuelve el AI Agent:

```
{ "output": "la respuesta que armó la IA" }
```

Lo que devuelve mi aprobación (botón Approve/Decline):

```
{ "data": { "approved": true } }
```

Lo que se manda a Airtable cuando apruebo:

```
{ "Consulta": "{{ snippet del mail }}", "Historial de Cambios": "{{ respuesta de la IA }}" }
```

4. Qué modelo usé en cada parte, y por qué

Como todavía es un proyecto chico y de práctica, no usé varios modelos distintos, pero sí pensé la decisión como me pidieron en la consigna: qué usaría en cada tarea si esto escalara de verdad.

Tarea	Qué usé / usaría	Por qué
Leer el mail y armar la primera respuesta (AI Agent)	Gemini 1.5 Flash	Es una tarea que se repite con cada mail que entra, así que necesito algo rápido y barato. No hace falta un modelo grande para escribir una respuesta de 300 caracteres.
Reformular cuando rechazo una respuesta (AI Agent2)	Gemini 1.5 Flash	Es prácticamente la misma tarea que la anterior, solo que con el feedback de que la primera no sirvió, así que no tiene sentido cambiar de modelo.
Casos que se escalan a mano (reclamos largos o complicados)	Un modelo más grande, tipo Claude	Estos no los interpretó el flujo automático, pero si tuviera que revisar un hilo largo con contexto complejo, ahí sí conviene pagar un modelo mejor, porque son pocos casos y no encarece todo el volumen.
Reprocesar muchos correos viejos de una	API de lotes (Batches)	Si algún día necesito reclasificar meses de historial en Airtable, no tiene sentido pagar el precio de una llamada en vivo por cada uno; el procesamiento por lotes sale bastante más barato.

La lógica general es: el modelo barato y rápido se banca el 100% del volumen automático, y el modelo caro queda reservado para las excepciones que decide escalar una persona.

5. Seguridad y qué pasa cuando algo falla

Qué datos toca el sistema

- El AI Agent solo recibe el snippet del mail (un fragmento corto), no el correo completo ni los adjuntos, para no exponerle de más a la IA.
- Las credenciales de Gmail y Airtable están guardadas como credenciales de n8n, no aparecen escritas en ningún nodo ni en el JSON que se exporta.
- En Airtable solo guardo la consulta y la respuesta aprobada, nada de tokens ni datos internos que no necesito.

Qué pasa si algo se rompe

- El AI Agent tiene activado el reintento automático (retryOnFail) y, si aun así falla, no frena todo el flujo: sigue por la salida de error hacia 'Get a message!'.
- El nodo If compara el valor de 'approved' con un tipo de dato estricto (booleano), para no confundir un texto ""true"" con el valor real true.
- Todavía me queda pendiente poner un límite de reintentos en el circuito de aprobación, para asegurarme de que nunca pueda quedar dando vueltas en loop.

El punto de aprobación humana

- Antes de que el cliente reciba cualquier respuesta, el nodo 'Send message and wait for response' frena el flujo y me pide que apruebe o rechace desde un mail con dos botones.
- Si rechazo, no sale nada hacia el cliente: el AI Agent2 reformula y el caso vuelve a pasar por el mismo control. La decisión final siempre la toma una persona, nunca la IA sola.

6. Lo que todavía me falta: el dashboard

Esto es lo único que no llegué a cerrar del lado de Airtable:

- Entrar a la tabla 'Clientes', ir a Compartir y sincronizar → Compartir vista, y generar el link de solo lectura de la vista agrupada por 'Estado de Consulta'.
- Pegar ese link en el README del repositorio, junto con el link de solo lectura a toda la base, para que se pueda ver de un vistazo cuántos casos están pendientes, en proceso o respondidos.